Ask for Chiron: the add-on guide

How it works, how to add it to a Chiron you already run, and how to take it out again. Written for the person who looks after a Chiron deployment.

chiron-mcp, `askpatch/`, 2 November 2026

1. What Ask is

Ask is a tab in Chiron's interface where a person types a question in plain words, such as *How many patients have type 2 diabetes and were started on metformin within 90 days of the diagnosis?*, and gets an answer with Chiron's own figures. Follow-up questions keep the earlier patients in mind, answers can be broken down and charted, and one button loads the exact filters behind an answer into Chiron's query builder, where the person carries on as usual.

It is an **add-on**. Ask is a separate service that a deployment adds beside its Chiron when it wants it, and removes the same way. Chiron's code, settings and databases are not modified. Three things are added: one container, one route in the web server that already serves Chiron, and one switch in the interface build. Take the three away and Chiron is as it was.

Two rules hold for every answer. **Every number comes from a Chiron query** made as the person who asked, with their own Chiron permissions: the model never estimates. And **the model can only call Chiron**: it runs with no shell, no file access and no web access, only the Chiron query tools, so nobody can talk it into anything else.

2. How it works

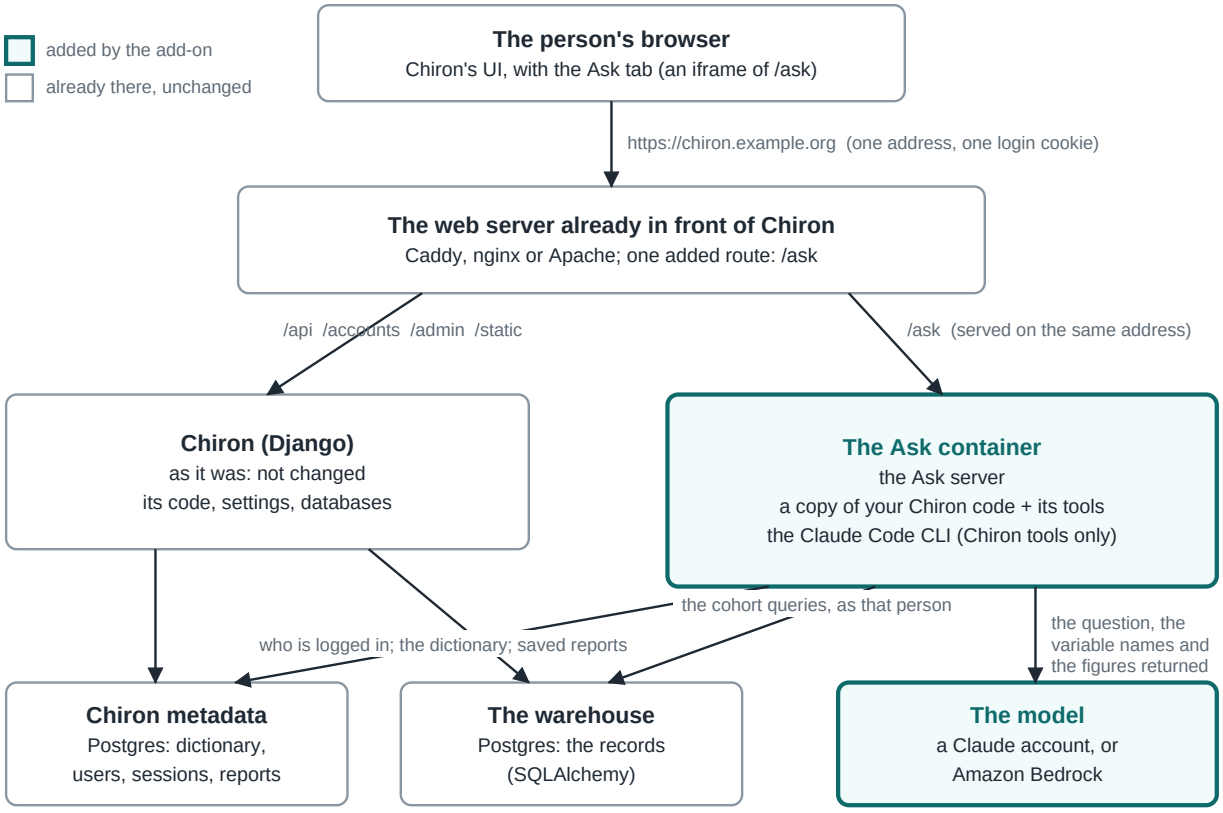


Figure 1. The pieces, and what travels between them. The teal pieces are what the add-on adds; the grey ones are the deployment's, unchanged.

The pieces

Piece	What it does	Added by the add-on?
Chiron	The Django application, its two Postgres databases, and the web server in front of it: all as before.	No
The /ask route	One snippet in that web server sends everything under /ask to the Ask container. Ask is then on Chiron's own address, which is what lets the browser send it the Chiron login.	Yes, one snippet (Caddy, nginx or Apache)
The Ask container	The Ask server; a copy of your Chiron's code, built from your checkout, so its processors and dependencies are yours; chiron-mcp's Chiron tools; and the Claude Code command-line client that runs each question.	Yes, one container
The Ask tab	A page in the interface that embeds Ask for the dataset being viewed. It is built in only when the build is told where Ask is (VITE_ASK_URL); otherwise the interface is exactly as before.	Yes, one build switch
The model	Claude, through a Claude account the container is logged in to, or through Amazon Bedrock in your AWS account. Which models the page offers is a line in the configuration.	Configuration

What happens when someone asks

1. The person is logged into Chiron and opens the Ask tab of a dataset. The tab shows the Ask page, served at `/ask` on Chiron's own address, so the browser sends Chiron's login cookie along with it, as it does for every Chiron page.
2. Ask looks the cookie up in Chiron's own session records and learns who is asking. No login, no answer: there is no service account to fall back to.
3. Ask starts a run of the model for that question, as that person. The model gets the dataset's variables and the Chiron tools, nothing else. The person's Chiron access level (aggregate, de-identified or identified) applies, capped by a ceiling in the configuration, de-identified by default.
4. The tools build the cohort with Chiron's own query engine and count it against the warehouse. Each figure in the answer is the result of such a call. A follow-up reuses the patients behind the earlier answer.
5. The answer streams back to the page as it is written. Under it: **Query · N** (the filters behind the answer, recounted as the person before the button appears; one click puts them in the query builder), **Report** or **Results** when the answer saved or changed one, and **Traces**, which shows what happened: each tool call, how long it took and which model answered.

What leaves the machine

The question, the names of the dataset's variables, and the figures and rows the tools return go to the model's provider: Anthropic for a Claude account, or your own AWS account for Bedrock. With synthetic data that is fine; with real data, check your rules and set the ceiling accordingly. Everything else, the databases included, stays where it is.

Two things the design needs

- **Ask must be on Chiron's address.** A browser sends a login cookie only to the host that set it. A Chiron whose interface is served from another host name than its API cannot carry the login to Ask; the route is not optional.
- **Chiron's metadata must be in Postgres** for the container. A container and the host cannot share SQLite's file locks safely, and reads then fail at random. A deployment that keeps its metadata in a SQLite file runs Ask on the host instead (section 7).

Two fixes Chiron needs, carried inside Ask

Ask relies on two corrections to Chiron 6.5.4: a mutable default argument in the cohort entry builder, and the relative-date rule ("within 90 days of") reading event dates that a dataset keeps in lookup tables. Your Chiron will not have them until they are merged upstream, so Ask applies both inside its own process when it starts and finds them missing, and says so in its log. Chiron's files are not written to, and Chiron's own query builder keeps its behaviour. A deployment that wants the query builder fixed too applies `askpatch/chiron-fixes.patch` to its checkout.

3. Before you start

- Chiron 6.5 on Postgres, with its metadata in Postgres as well.
- On the machine that will run Ask: the Chiron source checkout, the Django project (the directory with `manage.py`, its settings module and custom processors), and the two databases reachable from a container there.
- A web server in front of Chiron that can take one more route: Caddy, nginx or Apache.
- Docker with Compose 2.17 or newer.
- A model: a Claude account (Pro or Max) to log the container in to, or Bedrock with the model ids allowed for the machine's role or an AWS profile.
- For the Ask tab: the way this deployment builds its interface (the React UI with npm), or the built-in Django pages.

4. Adding it, step by step

4.1 Get the code and describe your Chiron

```
git clone https://github.com/ri-ds/chiron-mcp.git
cd chiron-mcp/askpatch
cp env.example .env && chmod 600 .env
```

Fill in `.env`. It stays on this machine (git ignores it) and holds the database passwords, so keep it readable by you only.

Line	What to put
CHIRON_SRC	The Chiron checkout: the directory holding `chiron/` and `requirements/`. The image is built from it.
CHIRON_PROJECT_DIR	The Django project directory (with manage.py). Mounted read-only into the container. CHIRON_SETTINGS names the settings module if it is not `project.settings`.
CHIRON_MCP_METADATA_DB	The metadata database, as a `postgresql://` URL **as the container reaches it**: a Postgres on this same machine is `host.docker.internal`, not localhost.
CHIRON_MCP_WAREHOUSE_URL	The warehouse, the same way: what CHIRON_SQL_ALCHEMY_CONNECTION_STRING says, written for the container.
PUBLIC_URL	Where people open Chiron, with its path prefix if it has one, for example `https://chiron.example.org` or `https://example.org/chiron`.
ASK_BIND	The port the web server forwards /ask to; `127.0.0.1:8900` unless changed.
CHIRON_MCP_MAX_ACCESS_LEVEL	The ceiling on what Ask may see: agg, deid (the default) or phi.
CHIRON_MCP_ASK_MODELS	The models the page may switch between, as `id=Label` pairs separated by commas. Unset: the model the client is set to, and no menu.

Line	What to put
The model lines	For Bedrock: CLAUDE_CODE_USE_BEDROCK=1, AWS_REGION, ANTHROPIC_MODEL, and AWS_PROFILE when the credentials come from a profile. For a Claude account: nothing; you log the container in once (4.5).

Everything in `.env` reaches the container. If your settings module reads an environment variable of its own, add that line too. Python packages your project needs beyond Chiron's requirements go in `requirements-extra.txt` before the build.

4.2 Start the container

```
docker compose up -d --build
docker compose ps                  # "healthy" after a few seconds
docker compose logs ask            # ends with "Chiron Ask running at http://localhost:8900"
```

The build installs your Chiron's requirements, copies its code in, adds chiron-mcp and the Claude Code client, and takes a few minutes the first time. The log also names the two fixes it applied (section 2), one line each, when your Chiron lacks them. The container listens on this machine only, on the port in ASK_BIND; nothing else is published.

4.3 Add the route to the web server

Put the snippet for your server inside the site that serves Chiron, then reload it. The snippets are in `askpatch/proxy/`; this is what they say.

Caddy, inside the site block, before its catch-all:

```
redir /ask {http.request.orig_uri.path}/ 308
handle_path /ask/* {
    reverse_proxy 127.0.0.1:8900 {
        flush_interval -1        # answers stream as events; pass each one on at once
    }
}
```

nginx, inside the server block, before a catch-all location:

```
location = /ask { return 308 /ask/; }
location /ask/ {
    proxy_pass http://127.0.0.1:8900/;      # the trailing slash strips /ask
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;                     # answers stream as events
    proxy_cache off;
    proxy_read_timeout 300s;
}
```

Apache httpd (mod_proxy_http), in the virtual host, before any broader ProxyPass:

```
RedirectMatch 308 ^/ask$ /ask/
ProxyPass        /ask/ http://127.0.0.1:8900/ timeout=300 retry=0
ProxyPassReverse /ask/ http://127.0.0.1:8900/
```

If Chiron is served under a path prefix, such as `/chiron`, the route is `/chiron/ask`. The streaming line matters: without it some servers hold the whole answer back and the page shows nothing until the end.

4.4 Turn the tab on

The React interface (is4r-chiron-ui). The tab is a build-time switch: a build made with `VITE_ASK_URL=/ask` has it, a build without the variable is as before. A checkout that does not have the switch yet gets it from a patch of one file:

```
cd is4r-chiron-ui
git apply /path/to/chiron-mcp/askpatch/ui-ask-tab.patch     # or: patch -p1 < ...
VITE_ASK_URL=/ask npm run build                              # then deploy the build as usual
```

The built-in Django pages. Two lines in the deployment's settings.py, after TEMPLATES is defined, put an Ask link in Chiron's own navigation bar (a settings_custom.py cannot see TEMPLATES directly; the template file's comment has the form for it):

```
TEMPLATES[0]["DIRS"].append("/path/to/chiron-mcp/askpatch/templates")
CHIRON_EXTRA_NAVBAR_ITEMS = "askpatch/ask_navbar.html"
```

4.5 Connect a model

A Claude account. Once, in a real terminal, log the container in:

```
docker compose exec -it ask claude
```

Choose the subscription login, open the link it prints, approve it in your browser, paste the code back, then type `/exit`. The login is kept in the container's home volume, so this is once per machine. Every question then uses that account's limits.

Amazon Bedrock. Set `CLAUDE_CODE_USE_BEDROCK=1`, `AWS_REGION` and `ANTHROPIC_MODEL` in `.env`. On an EC2 instance the instance role is used; its metadata hop limit must be 2, or a container cannot reach it. Elsewhere, name an AWS profile and start with the extra file that mounts your credentials read-only:

```
docker compose -f compose.yml -f compose.aws.yml up -d
```

The role must be allowed to invoke each listed model; the first call to a model also subscribes the account to it, which needs the Marketplace actions in the policy. A model Bedrock refuses is dropped from the page's menu and the question is answered by the next one.

4.6 Check it

```
python3 check.py                       # or: python3 check.py --user alice --dataset <id>
```

The check goes through the same front door a browser uses. It makes a login session for the account inside the container, so no password is typed or sent, and prints one line per check:

Line	What it confirms
the Ask container answers	The container is up and healthy on its port.
Ask answers only people logged into Chiron	No fallback account is configured.
Chiron's login page	The site itself answers through PUBLIC_URL.
the Ask page through the site	The /ask route reaches the container.
a Chiron session for the user	The container can read Chiron's metadata and the account exists.
Ask knows who is logged in	The login cookie reaches Ask and is accepted: same address, same database.
the user can see a dataset	The account has access to at least one dataset.
Chiron counts the dataset	Chiron's own engine works inside the container, against the warehouse.
Ask answers a question with Chiron's count	A real question; the figure in the answer equals Chiron's count.
answers stream through the web server	The events arrived over the run, not all at once at the end.

Every line should say PASS. Then open Chiron, log in, open a dataset, and the Ask tab is there.

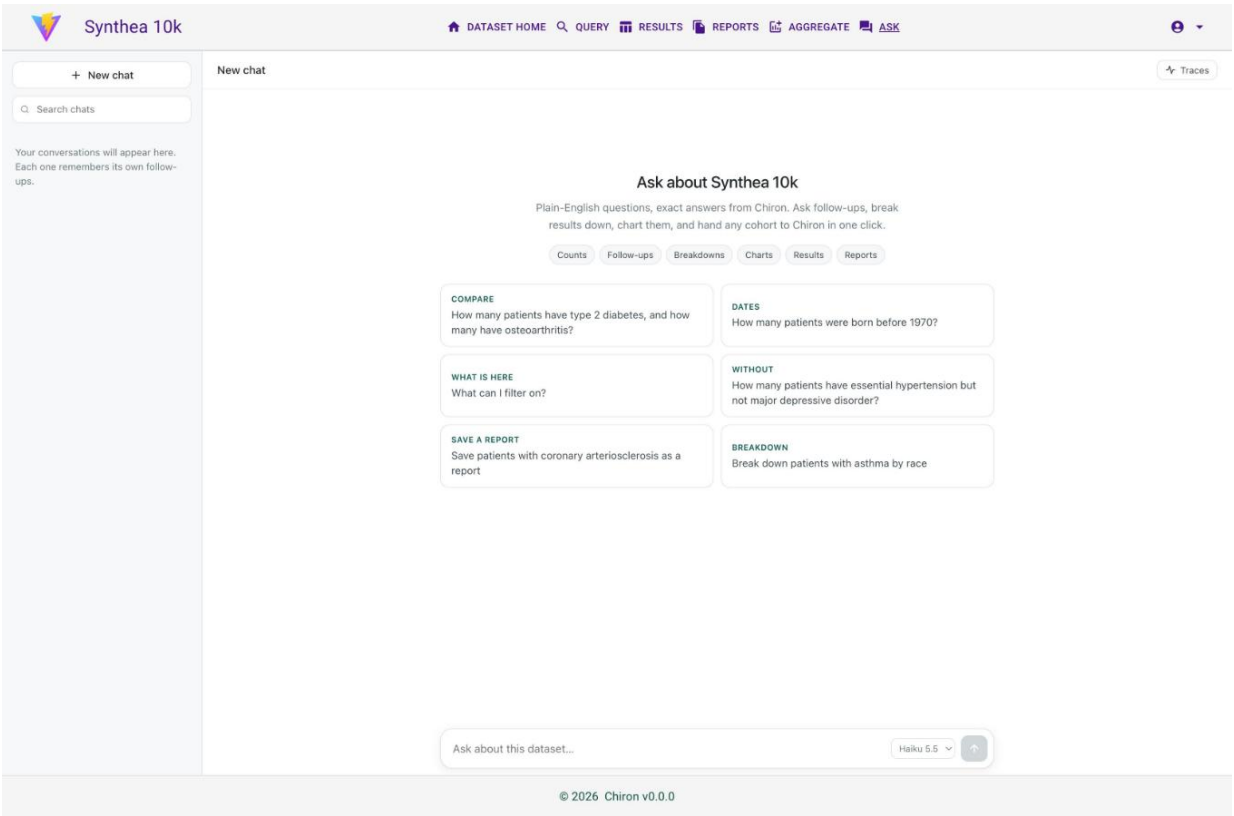


Figure 2. The Ask tab in a stock Chiron interface, served through the web server's /ask route, with the model menu at the bottom right and starter questions built from the dataset's own values.

5. For the people who use it

- **Ask in plain words.** Counts, combined filters, dates and timing between records, "without", comparisons, breakdowns by one or two variables, rankings, patient lists, saved reports, the Results tab. The starter cards show six questions the dataset can answer.
- **Follow up.** "Of those, how many are women?" refers to the patients behind the previous answer. New chat starts afresh; the sidebar keeps past conversations, per person and dataset.
- **Check the figures.** Query · N under an answer puts its exact filters into the query builder, where N is Chiron's count for them. Traces shows how the answer was produced.
- **Pick a model** in the menu by the question box when the deployment offers several; the trace names the one that answered.
- **Limits.** An answer takes ten to forty seconds. A conversation idle for twelve hours is closed and its transcript deleted; deleting a conversation in the sidebar deletes it at once.

6. Day to day

Task	How
See the log	`docker compose logs -f ask`
Restart	`docker compose restart ask`
Change the models or the ceiling	Edit `.env`, then `docker compose up -d`
Update Ask	`git pull` in chiron-mcp, then `docker compose up -d --build`
After upgrading Chiron	Rebuild the image the same way: it carries a copy of Chiron's code.
Where its data lives	The `ask-home` volume: the model login, conversation state and traces. Nothing of Ask's is in Chiron's databases except reports people chose to save, which are ordinary Chiron reports.
Who can use it	Anyone who can log into Chiron, with what Chiron lets them see, under the ceiling. There is no separate user list.

7. Running it on the host instead

A deployment that keeps Chiron's metadata in a SQLite file, or that does not run Docker, runs the same Ask server in its own Python environment: install chiron-mcp's dependencies there (its `constraints.txt` pins what was tested; SQLAlchemy must stay below 2.1), install Node 22 with the Claude Code client (`npm install -g @anthropic-ai/claude-code`), and start `python -m chiron_mcp.webapp` under systemd with the same variables as `.env` (CHIRON_MCP_REQUIRE_SESSION=1, CHIRON_MCP_ALLOW_SAVE=1, CHIRON_MCP_UI_URL set to the site). The route and the tab are the same. Keep the Ask process on the machine that holds the SQLite file.

8. Taking it out

1. `docker compose down -v` in `askpatch/`: removes the container, its model login and its conversations.
2. Delete the /ask snippet from the web server and reload it.
3. Rebuild the interface without `VITE_ASK_URL` (or `git apply -R ui-ask-tab.patch`), or remove the two navbar settings lines.

Chiron's code, settings and databases were never changed, so there is nothing there to undo. Reports people saved through Ask remain, as any saved report would.

9. When something goes wrong

What you see	What it means, and what to do
`docker compose up` stops with "set CHIRON_SRC in askpatch/.env"	`.env` is missing or incomplete. Copy env.example and fill it in.
The build fails in pip with a version conflict	Your Chiron's pins differ from chiron-mcp's constraints. Put USE_CONSTRAINTS=0 in .env and rebuild.
The container keeps restarting; the log shows a KeyError or ImportError inside your settings module	Your settings read a variable or import a package the container does not have. Add the variable to .env, or the package to requirements-extra.txt, and rebuild.
"connection refused" or "could not translate host name" for a database	The URL is written for the host, not the container: use host.docker.internal or the database's network address, and check that Postgres accepts connections from it.
/ask on the site gives 502 or 404	The route is missing or points at the wrong port, or the container is down: `docker compose ps`.
The Ask page says "Log in to Chiron first" although you are logged in	The cookie is not reaching Ask: Ask is not on Chiron's address, or the metadata database in .env is not the one Chiron writes its sessions to.
"Ask is not connected to a Claude account"	No model yet: `docker compose exec -it ask claude`, or the Bedrock lines.
Bedrock refuses with 403, or "not authorized to perform the required AWS Marketplace actions"	The role lacks invoke or Marketplace rights for that model, or the instance metadata hop limit is 1.
The answer appears all at once at the end	The web server buffers the stream. Add the streaming line from the snippet (proxy_buffering off; flush_interval -1).
"Timing between records needs an event date ... this dataset gives them none"	That table has no event date field in the data dictionary. Set it in the Django admin (the collection's event date field), and the timing questions work.
No Ask tab after the build	VITE_ASK_URL was not set for the build, or the patch was not applied to that checkout.

Appendix. Everything .env can hold

Variable	Default	Meaning
CHIRON_SRC	required	The Chiron checkout the image is built from
CHIRON_PROJECT_DIR	required	The Django project, mounted read-only
CHIRON_SETTINGS	project.settings	The settings module
CHIRON_MCP_METADATA_DB	required	Metadata Postgres URL, as the container reaches it
CHIRON_MCP_WAREHOUSE_URL	required	Warehouse Postgres URL, as the container reaches it
PUBLIC_URL	required	Where people open Chiron, prefix included
ASK_BIND	127.0.0.1:8900	Host address and port the web server forwards /ask to
CHIRON_MCP_MAX_ACCESS_LEVEL	deid	Ceiling: agg, deid or phi
CHIRON_MCP_ASK_MODELS	unset	Models offered on the page, id=Label pairs
CHIRON_MCP_THREAD_TTL_HOURS	12	Idle hours before a conversation is closed and deleted
CHIRON_MCP_TURN_SECONDS	240	Longest an answer may run
CHIRON_MCP_PG_CHECK_INTERVAL	2000	Postgres client check interval (0 for a Postgres on macOS)
CLAUDE_CODE_USE_BEDROCK, AWS_REGION, AWS_PROFILE, ANTHROPIC_MODEL	unset	Bedrock instead of a Claude account
CLAUDE_CODE_VERSION	2.1.283	The Claude Code client version the image installs
USE_CONSTRAINTS	1	0 lets your Chiron's own version pins win
ASK_CHECK_USER, ASK_CHECK_DATASET	demouser, first seen	What check.py signs in as and asks about

What was proven before this guide was written: the add-on installed as above against an unmodified Chiron 6.5.4 checkout with its own settings, the stock interface built with the switch, and Caddy in front with the snippet. The container started on that Chiron's settings and applied the two fixes; the check passed every line but the model step, which waits for the operator's login; the timing, safety and smoke test suites passed inside the container; and the tab and the logged-in identity showed in the browser (Figure 2). The record is in `docs/plan-askpatch.md` in the repository.